

A Survey of Program Repair and Automated Debugging with Code LLMs

InteractiveSurvey

Abstract

Automated Program Repair (APR) has evolved from rigid search-based techniques to flexible systems powered by Large Language Models (LLMs), which offer unprecedented capabilities in code understanding and reasoning. This survey comprehensively examines the systemic frameworks and methodological innovations that enable LLMs to function as autonomous debugging agents, moving beyond simple generative tasks. We propose a novel taxonomy categorizing approaches into agentic collaborations, iterative feedback mechanisms, and advanced context retrieval strategies, highlighting how role specialization and consensus protocols mitigate hallucination while enhancing transparency. Furthermore, the analysis reveals that execution-guided self-correction cycles and dynamic hypothesis generation significantly improve repair accuracy on complex logical errors by transforming debugging into an evidence-based exploration process. Finally, we identify critical trade-offs between computational cost and verification rigor, outlining key challenges in arbitration efficiency and standardized evaluation for future research. This work serves as a foundational reference for developing reliable, scalable, and intelligent debugging systems capable of addressing the semantic complexities of real-world software defects.

1 Introduction

Automated Program Repair (APR) has long been a cornerstone of software engineering research, aiming to reduce the substantial manual effort required to identify and fix software defects [1]. Traditional approaches have predominantly relied on search-based techniques, constraint solving, or statistical machine learning models trained on historical bug-fix pairs. While these methods achieved

notable success on specific benchmarks, they often struggle with the semantic complexity of real-world bugs, limited generalization across diverse codebases, and an inability to understand broad project contexts. The emergence of Large Language Models (LLMs) trained on massive code corpora has fundamentally altered this landscape, offering unprecedented capabilities in code understanding, generation, and reasoning [2]. These models promise to overcome the rigidity of earlier systems by leveraging deep semantic knowledge and flexible natural language interfaces to address intricate debugging challenges that previously required human intuition.

This survey paper provides a comprehensive examination of the rapidly evolving field of program repair and automated debugging powered by Code LLMs [3]. Unlike previous reviews that focus primarily on model architectures or benchmark performance, this work specifically investigates the systemic frameworks and methodological innovations that enable LLMs to function effectively as autonomous or semi-autonomous debugging agents [4]. We explore how the integration of agentic workflows, iterative refinement loops, and advanced context management strategies transforms the repair process from a singular generative task into a sophisticated, multi-stage engineering workflow. By synthesizing recent advancements, this paper aims to clarify the current state-of-the-art, identify persistent bottlenecks, and outline the trajectory for future research in building reliable, scalable, and intelligent debugging systems.

The core of this survey analyzes the architectural shifts toward agentic and iterative debugging frameworks, which decompose complex repair tasks into manageable sub-problems. We examine multi-agent collaboration architectures where role specialization allows distinct agents to handle fault localization, root cause analysis, and patch synthesis independently, thereby mitigating cog-

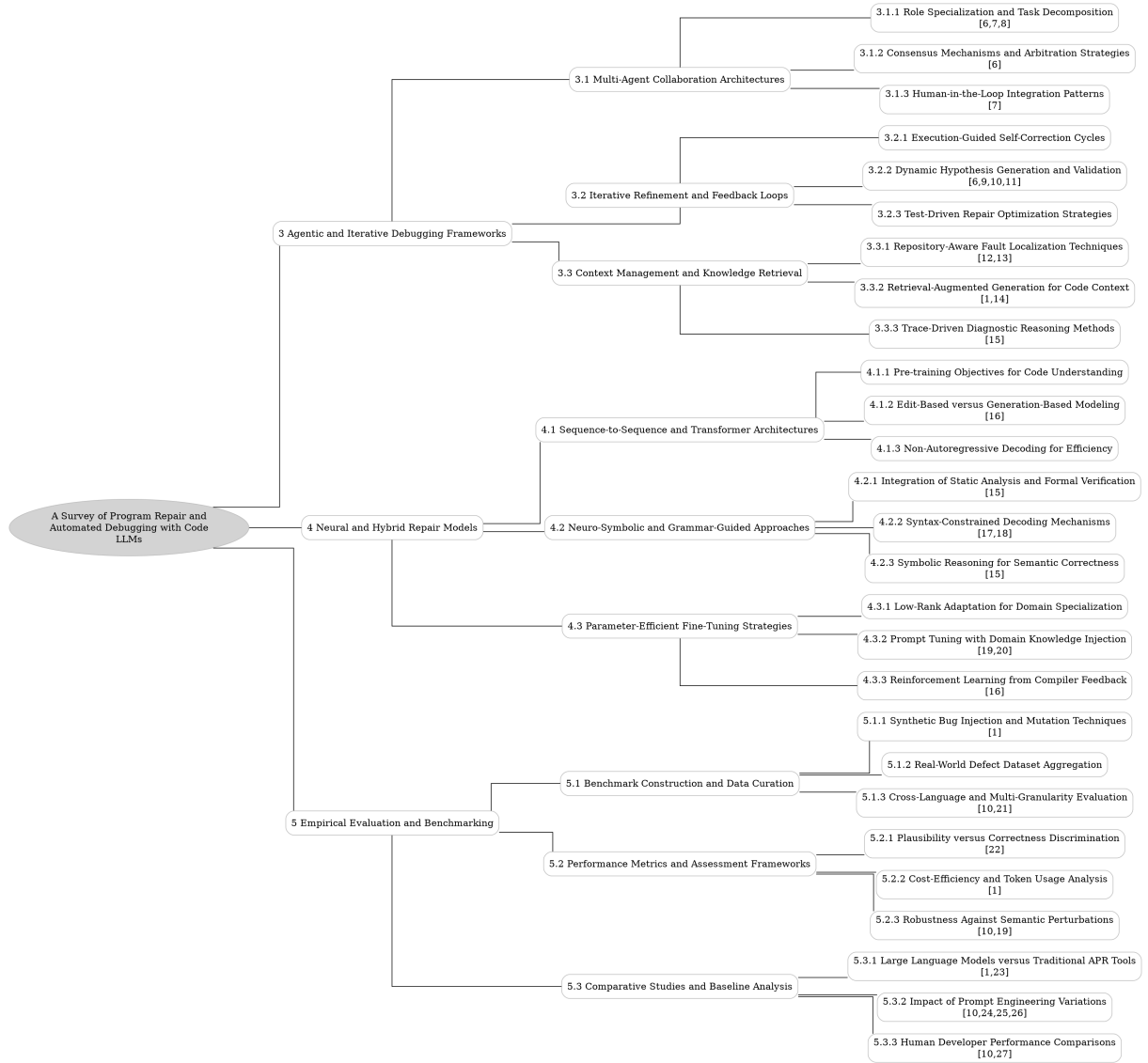


Figure 1: The outline of our survey: A Survey of Program Repair and Automated Debugging with Code LLMs

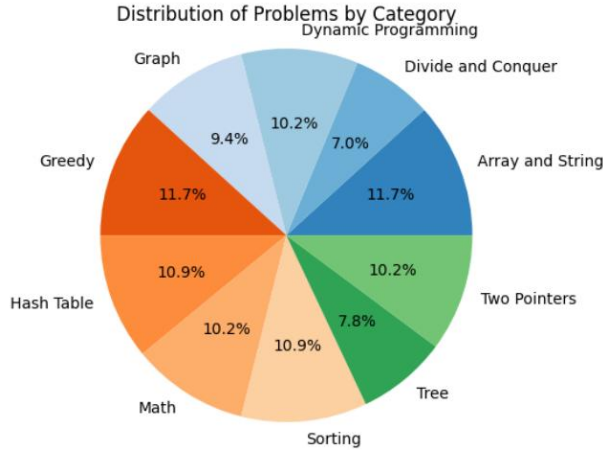


Figure 2: Chart from 'Extending the Frontier of ChatGPT: Code Generation and Debugging' (arXiv:2307.08260)

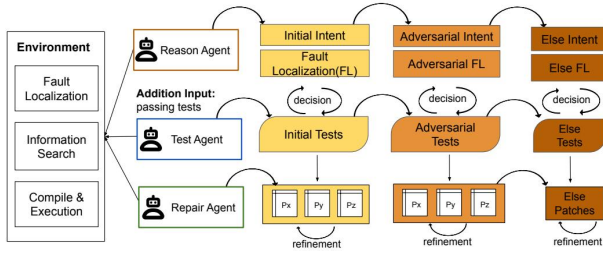


Figure 3: Chart from 'Adversarial Reasoning for Repair Based on Inferred Program Intent' (arXiv:2505.13008)

$$PR = \sum_{i=0}^n \frac{\bigwedge_{j=0}^{m_j} [a_{\theta_i^*}(x_i^j) = y_i^j]}{n} \times 100\%$$

Figure 4: Chart from 'Debugging with Open-Source Large Language Models: An Evaluation' (arXiv:2409.03031)

native overload. Furthermore, we discuss consensus mechanisms and arbitration strategies designed to reduce hallucination through multi-agent synergy and hierarchical judging, as well as patterns for integrating human expertise into these loops to resolve ambiguity and ensure alignment with domain constraints. These sections highlight how modular designs enhance transparency and controllability compared to monolithic model approaches.

Subsequently, the paper delves into the critical role of iterative refinement and feedback loops in enhancing repair accuracy. We detail execution-guided self-correction cycles where runtime errors and stack traces are fed back into the model to guide subsequent generation attempts, moving beyond static analysis to dynamic validation. The discussion extends to dynamic hypothesis generation, where LLMs emulate scientific debugging by formulating and testing explanations for bugs, and test-driven optimization strategies that leverage test suites to constrain search spaces and prevent overfitting [5]. These methodologies collectively represent a paradigm shift from blind search to guided, evidence-based exploration, significantly improving success rates on complex logical errors.

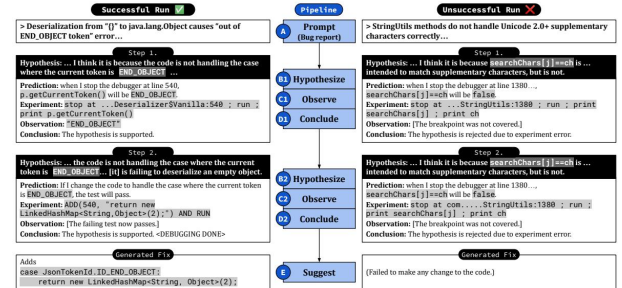


Figure 5: Chart from 'Explainable Automated Debugging via Large Language Model-driven Scientific Debugging' (arXiv:2304.02195)

Finally, we address the challenges of context management and knowledge retrieval, which are essential for handling large-scale, multi-file projects. This section reviews repository-aware fault localization techniques that utilize holistic project structures and knowledge graphs to identify buggy regions rather than isolated lines. We also explore retrieval-augmented generation strategies that enable models to access relevant documentation, dependency graphs, and historical fix patterns, ensuring that generated patches are consistent with the broader codebase semantics. By synthesizing these diverse technical threads, the

survey provides a structured understanding of how context awareness underpins the effectiveness of modern LLM-based repair tools [1].

The primary contributions of this survey are threefold. First, we propose a novel taxonomy that categorizes existing LLM-based repair approaches based on their architectural patterns, specifically distinguishing between agentic collaborations, iterative feedback mechanisms, and context retrieval strategies [6]. Second, we provide a critical analysis of the trade-offs inherent in these designs, such as the balance between computational cost and verification rigor in consensus mechanisms, or the tension between agent autonomy and human oversight. Third, we identify key open challenges and future research directions, including the need for more efficient arbitration protocols, robust handling of non-deterministic model outputs, and the development of standardized evaluation metrics for agentic debugging workflows, thereby serving as a foundational reference for researchers and practitioners in the field.

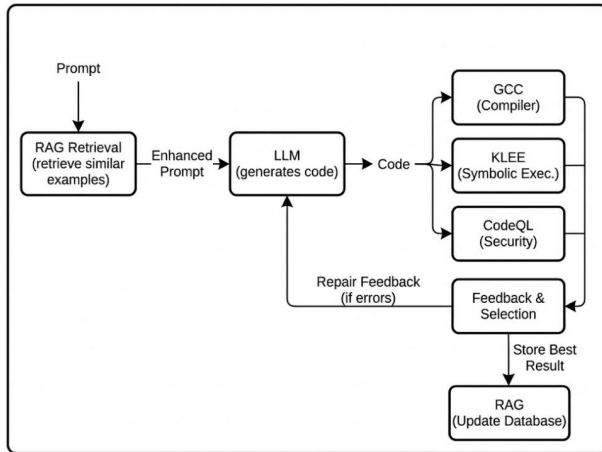


Figure 6: Chart from 'Improving LLM-Assisted Secure Code Generation through Retrieval-Augmented-Generation and Multi-Tool Feedback' (arXiv:2601.00509)

2 Agentic and Iterative Debugging Frameworks

2.1 Multi-Agent Collaboration Architectures

2.1.1 Role Specialization and Task Decomposition

Role specialization and task decomposition represent a paradigm shift in automated program repair, moving away from monolithic models toward modular, agent-based architectures [7]. By decomposing the complex repair workflow into distinct

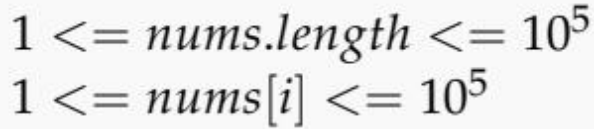
sub-tasks such as fault localization, root cause analysis, and patch synthesis, systems can assign specialized agents to each phase. This separation of concerns allows individual components to optimize for specific objectives, such as maximizing recall during slicing or precision during code generation, thereby mitigating the cognitive overload often observed when single models attempt to handle end-to-end reasoning for intricate bugs.

$$P(c) = \frac{\deg(c)}{\sum_{c_i \in C} \deg(c_i)}.$$

Figure 7: Chart from 'SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs' (arXiv:2504.14757)

The implementation of this strategy frequently relies on structured orchestration mechanisms, including task graphs and multi-agent collaboration protocols. In these frameworks, a planner agent typically decomposes high-level repair goals into executable steps, while specialized workers handle domain-specific operations like static analysis, dependency tracing, or test case verification. This modular approach facilitates the integration of diverse tools and modalities, enabling the system to leverage static program slices for context reduction while simultaneously employing large language models for semantic reasoning. Consequently, the repair process becomes more transparent and controllable, as errors can be isolated to specific stages rather than propagating through an opaque black box.

Furthermore, role specialization enhances the scalability and adaptability of repair systems across varying levels of task difficulty. By decoupling reasoning from acting, these architectures support iterative refinement loops where verification agents provide feedback to synthesis agents, effectively implementing a self-correction mechanism without requiring extensive model retraining [6]. This design not only improves the success rate on complex, multi-file issues but also allows for the seamless incorporation of human-in-the-loop interventions at critical decision points. Ultimately, decomposing the repair trajectory into specialized roles transforms the debugging process from a singular generative challenge into a coordinated, systematic engineering workflow [8].



```
1 <= nums.length <= 105
1 <= nums[i] <= 105
```

Figure 8: Chart from 'VIBEPASS: Can Vibe Coders Really Pass the Vibe Check?' (arXiv:2603.15921)

2.1.2 Consensus Mechanisms and Arbitration Strategies

Consensus mechanisms in LLM-based automated program repair primarily rely on multi-agent synergy and self-consistency protocols to mitigate hallucination and enhance fix reliability [6]. Approaches often employ a "majority rule" strategy where multiple agents independently generate patches, with the most frequently occurring solution selected as the candidate. More sophisticated frameworks utilize iterative discussion loops, allowing agents to critique and refine each other's outputs until a stable agreement is reached. While these methods significantly improve semantic correctness compared to single-model generation, they inherently incur higher computational costs due to excessive interaction rounds and token consumption during the convergence phase.

To address persistent disagreements where standard voting fails, hierarchical arbitration strategies are integrated into the workflow. These systems typically designate a superior model or a specialized "judge" agent to act as a final arbiter for disputed cases. In human-in-the-loop settings, this role is often fulfilled by independent researchers who conduct double-blind reviews of plausible patches, intervening only when initial assessments diverge. The arbitrator moderates the discussion or provides an independent analysis based on rigorous criteria, such as semantic equivalence or adherence to formal specifications, ensuring that the final judgment represents a well-considered consensus rather than a simple statistical majority.

Despite their effectiveness, current consensus and arbitration protocols face challenges regarding scalability and cost-efficiency. The reliance on powerful closed-source models for arbitration or extensive multi-round debates can lead to prohibitive latency and financial overhead, particularly in large-scale deployment scenarios. Furthermore, there is a risk that agents may uncritically accept misleading feedback during the consensus process, potentially reinforcing errors rather

than correcting them. Future iterations of these mechanisms aim to optimize the balance between verification rigor and resource utilization, possibly through adaptive termination criteria that halt interactions once a confidence threshold is met, thereby reducing average costs per resolved issue without compromising repair accuracy.

2.1.3 Human-in-the-Loop Integration Patterns

Human-in-the-loop integration patterns in automated program repair primarily manifest through iterative feedback cycles where developers validate or refine agent-generated patches [7]. These workflows often employ a Thought-Action-Observation structure, allowing the system to present reasoning traces and candidate fixes for human scrutiny before execution. By interleaving automated synthesis with manual verification, these patterns mitigate the risk of infinite reasoning loops and ensure that complex bug fixes align with implicit domain constraints that static analysis might miss.

Beyond simple validation, advanced integration leverages conversational interfaces to disambiguate requirements and incorporate structured human guidance directly into the prompting mechanism. Developers can inject chain-of-thought primitives or correct misleading feedback history, enabling the model to adapt its search strategy dynamically. This collaborative approach transforms the repair process from a purely generative task into a co-pilot interaction, where human intuition guides the agent toward plausible solutions while the AI handles the computational burden of scanning large codebases and generating syntactic variations.

However, effective integration requires balancing the cognitive load on developers against the autonomy of the agent. While lightweight summaries and explanation-enhanced patches improve intelligibility and acceptance rates, excessive interaction rounds can incur high computational costs and lead to user fatigue. Consequently, optimal patterns restrict human intervention to critical decision points, such as confirming root cause analysis or selecting among competing patch candidates, ensuring that the system remains efficient while benefiting from human expertise to resolve ambiguous or novel software defects.

2.2 Iterative Refinement and Feedback Loops

2.2.1 Execution-Guided Self-Correction Cycles

Execution-guided self-correction cycles represent a paradigm shift in automated program repair, moving beyond static code analysis to leverage dynamic runtime feedback for iterative refinement. In this framework, Large Language Models generate candidate patches that are immediately validated against an existing test suite or specific execution traces. When a patch fails, the resulting error messages, stack traces, or assertion violations are captured and fed back into the model as contextual prompts. This closed-loop mechanism allows the system to diagnose the root cause of failures with greater precision than static hints alone, effectively guiding the model to adjust its reasoning and code generation strategies in subsequent iterations.

The efficacy of these cycles relies heavily on the structured integration of execution diagnostics into the prompt engineering process. Rather than simply appending raw error logs, advanced approaches synthesize execution traces into concise summaries or natural language explanations that highlight semantic discrepancies between expected and actual behaviors. To balance exploration with computational efficiency, some methodologies employ adaptive strategies, such as incrementally increasing sampling temperature across iterations or utilizing combinatorial testing to validate batch-generated patches in parallel. This ensures that the model explores diverse repair trajectories while maintaining strict constraints on resource consumption and latency.

Empirical evaluations demonstrate that execution-guided self-correction significantly outperforms single-shot generation and traditional search-based repair techniques, particularly for complex logical errors and multi-location bugs. By iteratively refining patches based on concrete failure symptoms, these systems achieve higher fix rates on standard benchmarks like Defects4J and MBPP. However, challenges remain regarding the potential for feedback loops to reinforce incorrect hypotheses if the initial error analysis is flawed. Consequently, recent advancements incorporate confidence scoring and hypothesis rejection mechanisms to filter out misleading execution signals, ensuring that the self-correction process converges toward semantically correct

solutions rather than overfitting to specific test cases.

2.2.2 Dynamic Hypothesis Generation and Validation

Dynamic hypothesis generation leverages Large Language Models to emulate scientific debugging by iteratively formulating and refining bug explanations [9]. Instead of relying on static patterns, these approaches prompt models to explicitly articulate hypotheses regarding fault localization and root causes, often utilizing Chain-of-Thought reasoning to structure the diagnostic process. By treating bug fixing as an iterative cycle of hypothesizing, predicting outcomes, and designing experiments, systems can generate targeted candidate patches that address specific semantic discrepancies rather than merely satisfying syntactic constraints [10].

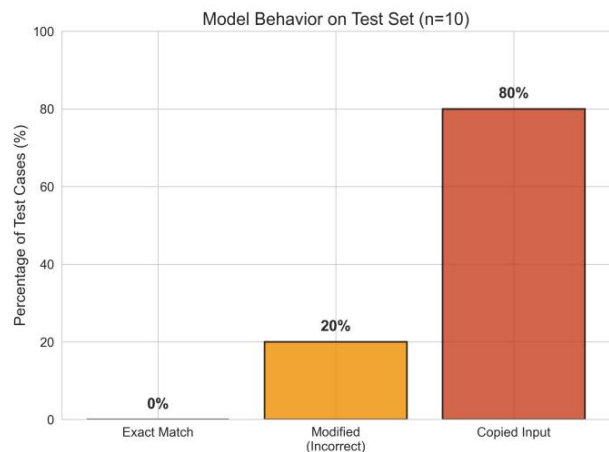


Figure 9: Chart from 'Syntax Is Not Enough: An Empirical Study of Small Transformer Models for Neural Code Repair' (arXiv:2512.22216)

The validation phase integrates dynamic execution feedback to rigorously test generated hypotheses against actual program behavior. When a candidate patch fails verification, the framework captures detailed failure symptoms, such as compilation errors or runtime exceptions, and feeds this information back to the model. This closed-loop mechanism allows the LLM to assess the consistency between its initial hypothesis and observed execution traces, effectively rejecting unsupported theories or refining them based on counterexamples [11]. Such dynamic filtering significantly reduces the search space by eliminating patches that do not align with the empirical evidence provided by the test suite.

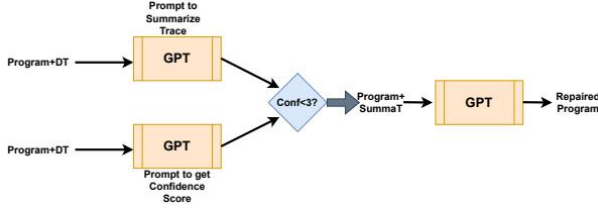


Figure 10: Chart from 'Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs' (arXiv:2505.04441)

To enhance reliability, advanced frameworks employ self-consistency mechanisms and multi-round interaction strategies to mitigate the stochastic nature of model outputs. By aggregating results from multiple reasoning paths or sampling attempts, these systems identify the most frequent and plausible error causes, thereby reducing bias in hypothesis selection. Furthermore, limiting the number of interaction rounds ensures a practical balance between computational cost and repair effectiveness, allowing the system to converge on a correct patch while avoiding infinite refinement loops. This dynamic interplay between generation and empirical validation represents a paradigm shift from traditional static analysis to adaptive, feedback-driven automated program repair [6].

2.2.3 Test-Driven Repair Optimization Strategies

Test-driven repair optimization strategies fundamentally shift the paradigm from blind search to guided exploration by leveraging test suites as dynamic feedback mechanisms. Traditional generate-and-validate approaches often suffer from inefficient search space traversal and test-data overfitting, where patches satisfy existing tests but fail to capture intended behavior. Modern methodologies address this by integrating test execution results directly into the repair loop, using failure information to constrain the search space and prioritize candidate patches that align with semantic correctness rather than mere syntactic validity.

Advanced implementations employ iterative validation frameworks where test outcomes dynamically influence subsequent generation steps. Techniques such as entropy-based ranking utilize test pass/fail signals to order patch candidates, significantly reducing the number of expensive test executions required to identify valid fixes. Furthermore, input minimization strategies automat-

ically reduce failure-inducing test cases to their core components, mitigating context overload for large language models and ensuring that repair attempts focus strictly on the logic responsible for the observed failures.

The integration of synthetic test generation further enhances these strategies by creating ensemble pass rate metrics that evaluate fix robustness across diverse scenarios. By treating generated tests as a synthetic suite, systems can assess the generalizability of potential patches before committing to full validation, effectively filtering out overfitted solutions. This cohesive combination of constraint synthesis, adaptive search ordering, and synthetic verification enables repair tools to achieve higher accuracy and efficiency, particularly in complex, multi-function repair scenarios where traditional heuristics often falter.

2.3 Context Management and Knowledge Retrieval

2.3.1 Repository-Aware Fault Localization Techniques

Repository-aware fault localization techniques represent a paradigm shift from traditional line-level spectrum-based methods to approaches that leverage holistic project context. Early explorations often decoupled localization from repair, relying on static analysis or simple similarity metrics that failed to capture intricate cross-file dependencies. Recent advancements address this by integrating Large Language Models (LLMs) with repository-level structures, enabling the identification of buggy regions rather than isolated statements [12]. This transition is critical for addressing multi-location bugs, where the root cause spans several files or functions, a scenario where conventional tools frequently struggle due to their inability to synthesize broad semantic relationships.

Modern methodologies employ hierarchical abstractions and knowledge graphs to enhance localization precision within large codebases. Techniques such as RepoRepair utilize LLMs to generate structured documentation ranging from function-level summaries to file-level overviews, creating semantic maps that guide the localization process. Similarly, approaches like FuseFL integrate traditional fault localization scores with LLM-generated reasoning, while others fine-tune code models specifically for direct line-level pre-

■ Smaug ■ Zephyr ■ Phind ■ DeepSeek-Coder
 ■ Mixtral ■ GPT-4

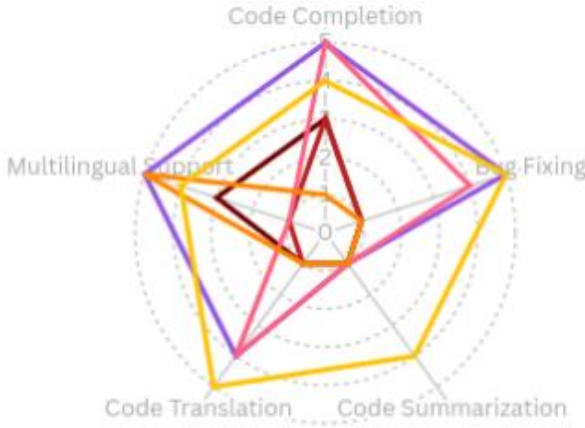


Figure 11: Chart from 'A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation' (arXiv:2411.07586)

diction within a repository context [13]. By encoding fault localization as a suspicious span or region instead of a single line, these methods provide a more realistic and robust foundation for subsequent repair stages, effectively mitigating the limitations of perfect localization assumptions.

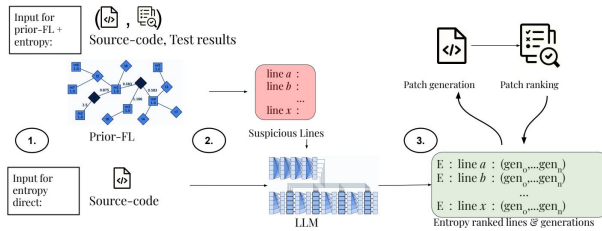


Figure 12: Chart from 'Revisiting Unnaturalness for Automated Program Repair in the Era of Large Language Models' (arXiv:2404.15236)

Despite these advances, challenges remain in balancing computational overhead with localization accuracy across diverse project architectures. The reliance on extensive context windows and the generation of hierarchical documentation introduce latency, necessitating efficient pruning strategies to maintain scalability. Furthermore, while repository-aware techniques significantly improve the identification of complex, distributed faults, their performance can still be influenced by the quality of available test suites and the completeness of the repository history. Future research

directions focus on optimizing context retrieval mechanisms and developing lightweight models capable of inferring deep semantic dependencies without exhaustive analysis, thereby enhancing the practical applicability of automated debugging in industrial settings.

2.3.2 Retrieval-Augmented Generation for Code Context

Retrieval-Augmented Generation (RAG) has emerged as a pivotal paradigm in Automated Program Repair (APR), addressing the inherent limitations of Large Language Models in handling long-context dependencies and rare bug patterns [1]. By integrating external knowledge bases, RAG-enabled systems like RAP-Gen leverage historical repair data to guide the generation process. These approaches typically identify code snippets with similar buggy contexts from vast repositories, allowing the model to adapt known solutions to current defects [14]. This mechanism is particularly effective because software bugs often exhibit recurring structural patterns, and referencing prior successful patches significantly enhances both the efficiency and accuracy of the generated fixes compared to generation-only models.

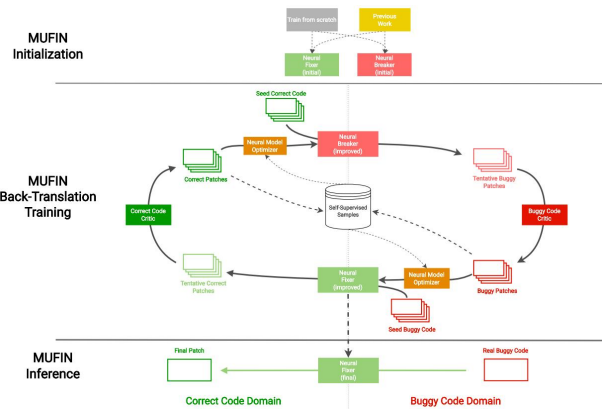


Figure 13: Chart from 'MUFIN: Improving Neural Repair Models with Back-Translation' (arXiv:2304.02301)

Beyond simple text similarity, advanced frameworks employ sophisticated code representation techniques to improve retrieval precision. Instead of relying solely on variable embeddings or raw token sequences, modern methods utilize Code Property Graphs (CPG) to capture multi-level dependencies across variables, methods, classes, and files. Tools such as Joern facilitate complex queries over these graphs, enabling the system to

gather semantically rich context that purely syntactic analyses might miss. Furthermore, some architectures incorporate iterative retrieval enhancement, where the system refines its search based on intermediate feedback or execution results, thereby dynamically narrowing the context to the most relevant code segments for the specific repair task at hand.

To mitigate the constraints of context window limits in large repositories, recent innovations propose using generated documentation as a proxy for retrieval. Rather than ingesting entire source files, these systems generate function-level or file-level summaries which serve as lightweight indices for locating relevant modules. Once the pertinent files are identified via text-based retrieval over these summaries, the detailed code is fed into the generator for patch synthesis. This two-stage process not only reduces computational overhead but also ensures that the generative model receives high-quality, focused context. Consequently, RAG for code context evolves from mere example copying to a nuanced integration of structural analysis, iterative refinement, and semantic summarization.

2.3.3 Trace-Driven Diagnostic Reasoning Methods

Trace-driven diagnostic reasoning methods leverage execution traces to bridge the gap between abstract failure symptoms and concrete root causes. Unlike approaches relying solely on static code analysis or outcome-level error messages, these techniques ingest runtime state information, such as variable values and control flow paths, to reconstruct the bug's manifestation. By analyzing divergence points where the actual execution trace deviates from the expected behavior, systems can pinpoint the precise location and nature of logical errors that static tools often miss, particularly in the absence of formal specifications.

To operationalize this, advanced frameworks integrate dynamic instrumentation with large language models to autonomously collect and interpret intermediate runtime evidence. These methods employ strategies like simulated instrumentation or deterministic rule-based tracing to capture method invocations, variable definitions, and file dependencies across the project. The retrieved trace data is then structured into a diagnostic context, enabling the model to perform step-by-step scientific debugging. This process allows the system to formulate hypotheses about internal ingre-

dients, such as specific variable misassignments or incorrect method calls, grounding the repair strategy in observed execution reality rather than probabilistic code patterns.

The integration of trace-driven reasoning significantly enhances the robustness of automated program repair by providing a transparent chain of logic for fault localization and patch generation [15]. By iteratively refining the understanding of the bug through execution feedback, these methods mitigate the limitations of models that rely exclusively on code semantics or weak oracles. Consequently, trace-driven approaches not only improve the accuracy of identifying subtle implementation-level errors but also facilitate the generation of explainable fixes, offering developers clear insights into why a specific patch was recommended based on the recorded execution history.

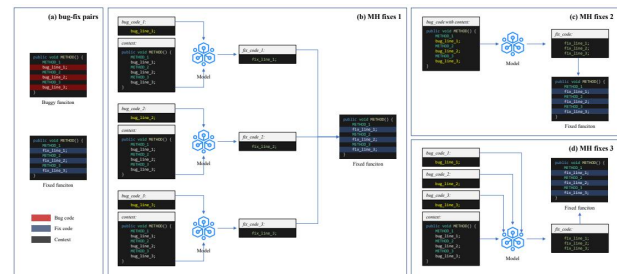


Figure 14: Chart from 'A Survey on Automated Program Repair Techniques' (arXiv:2303.18184)

3 Neural and Hybrid Repair Models

3.1 Sequence-to-Sequence and Transformer Architectures

3.1.1 Pre-training Objectives for Code Understanding

Pre-training objectives for code understanding have evolved from generic language modeling to specialized tasks that capture the unique structural and semantic properties of source code. Early approaches primarily adapted bidirectional encoder architectures using masked language modeling, treating code as a sequence of tokens similar to natural language. However, this method often fails to account for the strict syntactic constraints and logical dependencies inherent in programming languages, limiting the model's ability to infer deep semantic relationships or detect subtle defects without additional structural guidance.

To address these limitations, recent advance-

ments introduce identifier-aware pre-training strategies that explicitly model the role of variables, functions, and classes within the codebase. By distinguishing between identifier tokens and other lexical elements, these objectives enable models to learn consistent representation patterns for semantically equivalent symbols, even when their surface forms differ. This approach significantly enhances performance on tasks requiring precise variable tracking, such as bug localization and code clone detection, by aligning the learned embeddings with the underlying data flow and scope resolution rules rather than mere textual proximity.

Furthermore, unified pre-training frameworks now integrate both encoder and decoder components to support diverse downstream applications ranging from code summarization to automated program repair. These models employ multi-task learning objectives, including span denoising and contrastive learning, to simultaneously optimize for understanding and generation capabilities. By incorporating code-specific knowledge during the pre-training phase, such as data flow graphs and abstract syntax tree structures, these objectives ensure that the resulting representations are robust enough to handle complex reasoning tasks while maintaining high fidelity to the original program logic.

3.1.2 Edit-Based versus Generation-Based Modeling

Edit-based modeling approaches treat automatic program repair as a structured transformation task, predicting specific edit operations or sequences to modify buggy code rather than regenerating it entirely [16]. These methods often operate on abstract syntax trees or utilize specialized tokens to denote insertions, deletions, and replacements, thereby preserving the surrounding context and ensuring syntactic validity by construction. By focusing on minimal changes, edit-based models like Cedit leverage the structural integrity of the original program, which is particularly advantageous when dealing with localized bugs where the majority of the code remains correct. This paradigm reduces the search space significantly compared to full generation, as the model only needs to identify the precise deviation from the correct state.

In contrast, generation-based modeling frames repair as a sequence-to-sequence translation or in-

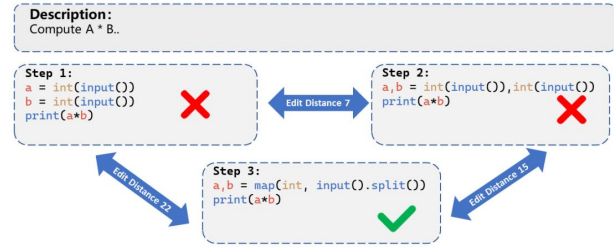


Figure 15: Chart from 'RePair: Automated Program Repair with Process-based Feedback' (arXiv:2408.11296)

filling problem, where the model synthesizes corrected code tokens directly from the buggy input and contextual information. Modern large language models employ this strategy using either left-to-right autoregressive decoding or bidirectional infilling mechanisms that condition on both prefix and suffix contexts. While generation-based approaches offer greater flexibility in handling complex, multi-line refactorings and do not require predefined edit schemas, they face challenges in guaranteeing syntactic correctness and often struggle with determining the appropriate termination point for the generated sequence. Furthermore, pure generative models typically lack access to suffix context in standard autoregressive settings, potentially limiting their ability to maintain coherence with subsequent code blocks.

The distinction between these paradigms ultimately influences the trade-off between precision and flexibility in patch synthesis. Edit-based models excel in scenarios requiring granular, minimal modifications but may be constrained by their inability to handle fragmented code snippets that fail to parse into valid structures. Conversely, generation-based models demonstrate superior scalability and language agnosticism, capable of producing diverse fixes without explicit edit constraints, yet they risk introducing unnecessary changes or hallucinating irrelevant code. Recent hybrid strategies attempt to bridge this gap by initially generating candidate token sequences and subsequently refining them through edit operations, aiming to combine the contextual awareness of generation with the structural rigor of edit-based refinement.

3.1.3 Non-Autoregressive Decoding for Efficiency

Non-autoregressive decoding (NAD) addresses the inherent latency of standard autoregressive

generation by producing all output tokens simultaneously rather than sequentially. In the context of automated program repair, where candidate patches must be synthesized rapidly, NAD eliminates the dependency on previously generated tokens during inference, theoretically reducing decoding time from linear to constant relative to the output length. This parallelization is particularly advantageous for large-scale codebases where iterative token prediction creates prohibitive computational bottlenecks, enabling real-time feedback loops that traditional sequential decoders cannot support.

To mitigate the accuracy degradation often associated with ignoring target-side dependencies, advanced NAD architectures incorporate auxiliary regularization and iterative refinement mechanisms. Techniques such as length prediction modules and knowledge distillation from autoregressive teachers allow the model to approximate joint token distributions without explicit sequential conditioning. By leveraging denoising objectives similar to those found in encoder-decoder pre-training, these models learn to reconstruct corrupted code spans in a single forward pass, effectively balancing the trade-off between generation speed and the syntactic coherence required for compilable software fixes.

Despite these efficiency gains, applying NAD to code generation presents unique challenges due to the strict structural constraints of programming languages. Unlike natural language, where minor grammatical errors may be tolerable, code requires precise token alignment and strict adherence to abstract syntax tree structures. Current research focuses on integrating explicit edit operations and constraint-based decoding strategies into the non-autoregressive framework to ensure that parallelly generated patches maintain logical consistency. These hybrid approaches aim to retain the throughput benefits of NAD while satisfying the rigorous correctness criteria necessary for deploying automated repair tools in production environments.

3.2 Neuro-Symbolic and Grammar-Guided Approaches

3.2.1 Integration of Static Analysis and Formal Verification

The integration of static analysis and formal verification creates a robust pipeline for automated

program repair by combining scalable defect detection with rigorous correctness guarantees [15]. Static analyzers, such as those utilizing abstract interpretation or data-flow analysis, efficiently identify potential violations and generate fix templates based on recurring patterns of semantic bugs. These tools serve as the initial filtering mechanism, pinpointing suspicious code locations and characterizing the nature of the error without the prohibitive computational cost associated with exhaustive formal methods. By leveraging warnings from static checkers, repair systems can constrain the search space to plausible modifications that address specific violation types, thereby enhancing the precision of candidate patch generation.

Once candidate patches are synthesized, formal verification techniques are employed to mathematically validate their correctness against specified properties or reference implementations. This phase often involves symbolic execution or satisfiability modulo theories (SMT) solvers to prove that the repaired program satisfies all preconditions and postconditions while eliminating the original fault. Unlike dynamic testing, which only covers executed paths, formal verification ensures that the fix holds for all possible inputs, effectively ruling out overfitting to specific test cases. The process verifies that the modified code preserves the intended semantics of the original program, ensuring that no new errors are introduced and that the solution aligns with the formal specification of the software’s behavior.

This synergistic approach addresses the limitations inherent in using either technique in isolation, balancing efficiency with soundness. While static analysis provides the necessary scalability to handle large codebases, it often suffers from false positives and an inability to guarantee semantic correctness. Conversely, formal verification offers strong correctness guarantees but struggles with state-space explosion in complex systems. By orchestrating these methods, modern repair frameworks first narrow down potential fixes using static insights and then subject only the most promising candidates to rigorous formal proof. This hybrid strategy significantly improves the reliability of generated patches, ensuring that repairs are not only syntactically valid but also semantically sound across all execution contexts.

3.2.2 Syntax-Constrained Decoding Mechanisms

Syntax-constrained decoding mechanisms address the critical challenge of ensuring syntactic validity in generated code by integrating formal grammar rules directly into the autoregressive generation process. Unlike standard neural machine translation approaches that rely solely on statistical likelihood, these methods leverage Abstract Syntax Trees (ASTs) to guide token prediction. By mapping the decoding space to valid production rules of a target programming language, the model is restricted from emitting token sequences that result in compilation errors, thereby significantly reducing the search space and improving the structural correctness of automated program repairs [17].

$$\hat{t} = \arg \max_t V(t).$$

Figure 16: Chart from 'Enhancing Automated Program Repair via Faulty Token Localization and Quality-Aware Patch Refinement' (arXiv:2511.18001)

A prominent implementation of this paradigm involves syntax-guided edit decoders, which replace traditional token-level generation with operation-based editing constrained by AST structures. In such architectures, the decoder predicts edit actions-such as insertion, deletion, or replacement of subtrees-rather than raw tokens, ensuring that every intermediate state remains a syntactically valid partial program. This approach often employs a provider-decider framework where the provider suggests valid edit candidates based on the current AST context, and the decider selects the optimal transformation, effectively bypassing the need for post-hoc syntax correction or specialized output designs that plague sequence-to-sequence models.

Recent advancements have further refined these mechanisms by incorporating two-stage decoding strategies to balance contextual understanding with syntactic rigor. The initial stage generates a coarse-grained fix to capture semantic intent, while the subsequent stage refines the output under strict grammar constraints to guarantee compilability. This hybrid methodology mitigates the limitations of non-autoregressive models that often struggle with long-range dependencies, allow-

ing the system to maintain global code coherence while adhering to local syntactic rules. Consequently, syntax-constrained decoding has become a foundational component in state-of-the-art neural program repair systems, bridging the gap between probabilistic language modeling and deterministic compiler requirements [18].

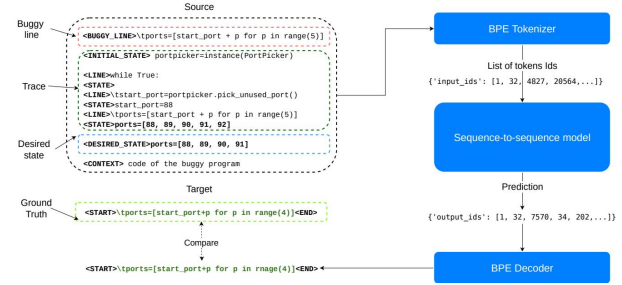


Figure 17: Chart from 'TraceFixer: Execution Trace-Guided Program Repair' (arXiv:2304.12743)

3.2.3 Symbolic Reasoning for Semantic Correctness

Symbolic reasoning serves as a critical mechanism for ensuring semantic correctness in automated program repair by formally verifying program behavior against specifications [15]. Unlike purely statistical approaches that rely on pattern matching, symbolic techniques employ methods such as symbolic execution and constraint solving to explore feasible execution paths and identify logical inconsistencies. This rigorous analysis allows repair systems to generate patches that are not only syntactically valid but also semantically equivalent to the developer's intent, effectively addressing deep logical errors that surface-level models often miss.

Recent advancements have integrated symbolic reasoning with neural architectures to leverage the strengths of both paradigms. Hybrid frameworks utilize symbolic analyzers to prune invalid candidate patches or guide the search space, ensuring that generated fixes adhere to strict logical constraints derived from the program's abstract syntax tree and data flow. By encoding structural dependencies and inter-word relationships into the reasoning process, these systems mitigate the risk of overfitting to test suites and reduce the incidence of semantically incorrect repairs that pass functional tests by chance.

Furthermore, symbolic reasoning enhances the interpretability and reliability of repair suggestions by providing formal guarantees of correct-

ness where possible. Approaches combining programming-by-examples with semantic guidance demonstrate superior performance in fixing complex bugs involving boolean logic, parameter ordering, and boundary conditions. As large language models increasingly incorporate reasoning-as-logic units, the fusion of neuro-symbolic methods promises to bridge the gap between probabilistic code generation and deterministic verification, ultimately yielding more robust and maintainable software fixes.

3.3 Parameter-Efficient Fine-Tuning Strategies

3.3.1 Low-Rank Adaptation for Domain Specialization

Low-Rank Adaptation (LoRA) has emerged as a pivotal parameter-efficient fine-tuning strategy for specializing large language models within the software engineering domain. By freezing the pre-trained weight matrices and injecting trainable low-rank decomposition matrices into each layer of the transformer architecture, LoRA significantly reduces the number of trainable parameters. This approach mitigates the substantial memory overhead associated with full-model fine-tuning, enabling the adaptation of massive code models on commodity hardware while preventing catastrophic forgetting of general linguistic and syntactic knowledge acquired during pre-training.

In the context of automated program repair and code generation, domain specialization via LoRA allows models to rapidly internalize specific bug patterns and fix semantics without retraining the entire backbone. Recent methodologies leverage this technique to align decoder-only language models with the specific objectives of patch generation, often utilizing synthetic benchmarks containing diverse mutation types to enhance robustness. The low-rank updates effectively capture the distributional shifts between natural language and source code syntax, facilitating the model's ability to generate compilable and semantically correct patches even when trained on limited domain-specific datasets.

Furthermore, integrating LoRA with retrieval-augmented generation frameworks enhances the precision of patch ranking and validation processes. By fine-tuning only the adaptation layers, systems can efficiently incorporate context from retrieved similar bugs or fix patterns, optimizing

the contrastive loss functions used to distinguish correct patches from incorrect ones. This modular adaptation not only accelerates the inference stage by maintaining a lightweight parameter footprint but also supports multi-task learning scenarios where a single base model is specialized for various software engineering tasks, such as defect categorization and transpilation, through distinct low-rank modules.

3.3.2 Prompt Tuning with Domain Knowledge Injection

Prompt tuning with domain knowledge injection represents a pivotal shift in automated program repair, moving beyond generic pre-training to explicitly embed software engineering semantics into the inference process. Unlike standard fine-tuning which updates all model parameters, this approach leverages parameter-efficient strategies to inject specific bug-fixing patterns, syntax constraints, and repair heuristics directly into the prompt structure. By incorporating retrieval-augmented contexts, such as relevant bug-fix pairs from external codebases or historical commit messages, models can access task-specific knowledge without requiring massive retraining. This mechanism allows large language models to bridge the gap between general code understanding and the precise logical requirements of patch generation, effectively mitigating the data scarcity issues often encountered in specialized repair domains [19].

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^4 w_n \log p_n \right)$$

Figure 18: Chart from 'Identifying and Mitigating API Misuse in Large Language Models' (arXiv:2503.22821)

The construction of these knowledge-enriched prompts typically involves a multi-component architecture that integrates natural language descriptions, buggy code snippets, and explicit fix patterns [20]. Advanced techniques employ few-shot learning where exemplar patches serve as in-context guides, steering the model toward syntactically correct and semantically valid modifications. Furthermore, recent methodologies introduce structured placeholders and provider-decoder architectures within the prompt to guarantee that generated tokens adhere to strict grammatical

rules. By embedding domain-specific constraints, such as variable scope limitations or API usage protocols, directly into the input sequence, these approaches significantly reduce the incidence of hallucinated code and ensure that the generated repairs maintain the original program's control flow and intent.

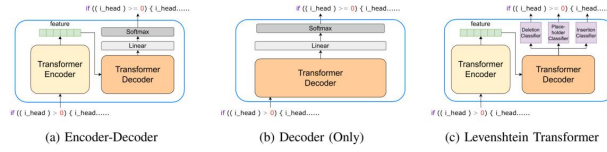


Figure 19: Chart from 'Automated Code Editing with Search-Generate-Modify' (arXiv:2306.06490)

Despite its promise, this paradigm faces challenges regarding the stability of instruction following and the optimal formulation of domain knowledge. The effectiveness of injected knowledge heavily depends on the precision of the prompt design; poorly structured constraints can lead to overfitting on specific bug types or failure to generalize across diverse defect categories. Current research addresses these limitations by iteratively refining prompt templates based on feedback loops involving test suite execution and syntax verification filters. Future directions suggest combining dynamic retrieval mechanisms with adaptive prompt engineering to create self-correcting systems that continuously update their injected knowledge base, thereby enhancing the robustness and applicability of prompt-tuned models in complex, real-world software maintenance scenarios.

3.3.3 Reinforcement Learning from Compiler Feedback

Reinforcement Learning from Compiler Feedback represents a paradigm shift in automated program repair, moving beyond static pattern matching to dynamic, execution-guided optimization. In this framework, the code generation process is modeled as a sequential decision-making problem where an agent iteratively refines buggy source code. Unlike supervised approaches that rely solely on human-written patch pairs, this method leverages the compiler as a deterministic oracle to provide immediate, high-fidelity feedback. The compiler validates syntactic correctness and semantic consistency, generating reward signals that guide the policy network toward compilable and functionally correct solutions, effectively narrowing the vast search space of potential edits.

The core mechanism involves a critic model that evaluates generated candidates based on compilation outcomes, such as error messages, warning counts, or successful build status. This feedback loop enables the system to perform execution-based backpropagation, where the loss function is directly influenced by the compilability of the generated code. By filtering out non-compilable candidates early in the training trajectory, the approach ensures that the neural fixer focuses on high-quality, self-supervised samples. Research indicates that integrating a compiler-centric critic significantly enhances repair effectiveness, allowing models to correct complex defects that purely data-driven methods often miss due to the scarcity of analogous training examples.

Furthermore, this strategy addresses the challenge of sparse rewards in code generation by providing dense, step-wise guidance during the learning process. The integration of compiler feedback allows the model to learn robust representations of syntax and type constraints without explicit annotation, fostering generalization across diverse programming languages and error types. Empirical studies demonstrate that models trained with compiler-derived rewards outperform baseline techniques in both patch compilability and overall repair success rates [16]. Consequently, reinforcement learning from compiler feedback has emerged as a critical component in modern repair frameworks, bridging the gap between probabilistic language modeling and the rigid logical requirements of software engineering.

4 Empirical Evaluation and Benchmarking

4.1 Benchmark Construction and Data Curation

4.1.1 Synthetic Bug Injection and Mutation Techniques

Synthetic bug injection serves as a critical mechanism for scaling training data in automated program repair, addressing the scarcity of real-world bug-fix pairs [1]. Drawing from mutation testing principles, these techniques systematically introduce faults into verified codebases to create labeled datasets where the original source acts as the ground truth. By treating existing repositories as correct versions and applying predefined error patterns, researchers can generate massive corpora of buggy-correct pairs, enabling the super-

vised fine-tuning of deep learning models without relying solely on the noisy signal of historical commit messages.

The implementation of these techniques varies from simple syntactic perturbations to sophisticated, semantics-aware transformations. Early approaches utilized random operator replacement or statement deletion, while modern strategies employ learned models to inject bugs that closely mimic human errors, such as incorrect comparison operators, variable misuse, or missing accessor calls. Advanced frameworks extend this to byte-code mutation and AST-level modifications, ensuring that generated defects trigger specific test failures while maintaining compilability. This granularity allows for the creation of diverse error distributions, covering both single-line slips and complex logical flaws that challenge localization capabilities.

Despite their utility in data augmentation, synthetic bugs face scrutiny regarding their fidelity to real-world software defects. While empirical studies suggest that mutants often overlap significantly with actual bug patterns, there remains a domain gap where models trained exclusively on artificial data struggle to generalize to intricate, multi-hunk production errors. Consequently, contemporary research focuses on hybrid approaches that combine synthetic injection with curated real-world benchmarks. These methods aim to balance the volume of generated data with the complexity of authentic defects, ensuring that repair models develop robust reasoning capabilities applicable to diverse and unpredictable software environments.

4.1.2 Real-World Defect Dataset Aggregation

The aggregation of real-world defect datasets primarily relies on mining version control systems to extract bug-fix pairs, preserving authentic variable names, method calls, and type information essential for training robust models. Prominent benchmarks like CodeXGLUE and Tufano utilize commit heuristics to harvest tens of thousands of Java method pairs from GitHub, creating large-scale corpora for neural machine translation approaches. However, this mining process inherently introduces noise, with approximately 5-8% of pairs exhibiting no semantic difference between buggy and fixed versions, often capturing unrelated changes or refactorings rather than genuine bug corrections. Despite these imperfections, researchers typically retain the full dataset

to maintain consistency with prior work and ensure fair comparative evaluations across different automated program repair techniques.

To address the limitations of mined data and enhance evaluation rigor, curated benchmarks such as Defects4J, BugsInPy, and BugsCpp provide verified real-world faults from diverse open-source projects. These datasets distinguish themselves by offering reproducible test suites that expose specific defects, allowing for precise validation of generated patches without relying solely on commit metadata. Recent efforts have expanded this landscape to include specialized subsets focusing on single-hunk bugs, performance issues, or multi-language contexts, thereby facilitating more granular analysis of repair capabilities. Furthermore, synthetic data augmentation strategies are increasingly employed to expand training corpora by injecting artificial bugs into clean code, significantly scaling dataset size while controlling for bug complexity and distribution.

The diversity and quality of aggregated datasets critically influence model generalization, prompting investigations into cross-dataset transferability and the impact of data leakage. Studies indicate that models fine-tuned on specific repositories may struggle when evaluated on unseen benchmarks, highlighting the necessity for diverse training sources that span multiple difficulty levels and project domains. Consequently, contemporary research emphasizes the construction of composite datasets that merge mined pairs with curated benchmarks to balance scale and verification accuracy. This aggregation strategy aims to mitigate biases inherent in single-source collections, ensuring that automated repair tools develop the capacity to handle the varied complexities found in actual software development environments.

4.1.3 Cross-Language and Multi-Granularity Evaluation

Traditional natural language metrics like BLEU often fail to capture the functional correctness of code, as they prioritize token-level similarity over semantic validity [10]. Consequently, execution-based metrics such as Pass@k have become the standard for assessing code generation, requiring robust test suites and sandboxed environments to verify behavior [21]. However, these approaches frequently overlook the nuances of cross-language generalizability, where models trained on dominant languages like Python may exhibit signif-

icant performance degradation when applied to less represented programming languages. Recent benchmarks emphasize the necessity of dynamically evaluating models across diverse linguistic ecosystems to accurately measure the gap between open-source and closed-source capabilities.

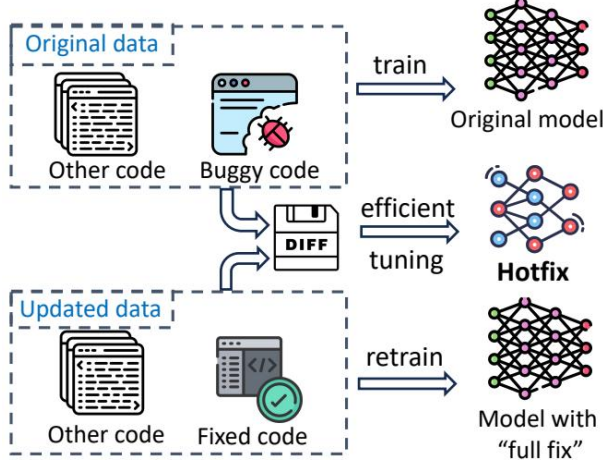


Figure 20: Chart from 'Hotfixing Large Language Models for Code' (arXiv:2408.05727)

Beyond language diversity, the granularity of evaluation plays a critical role in determining model efficacy. Studies indicate that performance varies substantially depending on whether the task is defined at the statement, method, or file level. For instance, while some models excel at completing single lines of code, they often struggle with maintaining contextual consistency across entire functions or modules. Evaluating repairability and generation accuracy at varying granularities reveals that providing finer-grained diagnostic information can significantly improve outcomes, yet many existing benchmarks remain limited to coarse-grained assessments that mask these structural dependencies.

To address these limitations, emerging evaluation frameworks advocate for multi-granularity and multilingual protocols that integrate syntactic analysis with deeper contextual understanding. Metrics like CodeBLEU attempt to bridge this gap by incorporating abstract syntax tree matching and dataflow similarity alongside n-gram overlap. Furthermore, comprehensive multitask evaluations are essential to isolate specific failure modes, such as syntactic bugs versus logical errors, across different programming paradigms. Future research must prioritize the development of unified benchmarks that simultaneously stress-test models on cross-language transferability and vary-

ing levels of code abstraction to ensure robust deployment in real-world software engineering scenarios.

4.2 Performance Metrics and Assessment Frameworks

4.2.1 Plausibility versus Correctness Discrimination

The distinction between plausibility and correctness represents a critical challenge in automated program repair, where a plausible patch satisfies existing test suites while a correct patch achieves semantic equivalence with a developer's ground truth [22]. Empirical evidence indicates that while large language models frequently generate syntactically valid and functionally plausible solutions, only a small fraction constitute actual correct fixes. For instance, studies reveal that despite high rates of test case passage, the proportion of patches exactly matching ground truth remains low, often below ten percent, highlighting a significant divergence between functional adequacy and true defect resolution.

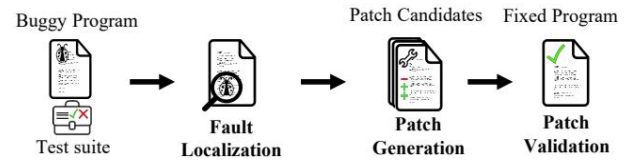


Figure 21: Chart from 'On the Efficiency of Test Suite based Program Repair: A Systematic Assessment of 16 Automated Repair Systems for Java Programs' (arXiv:2008.00914)

This discrepancy primarily stems from the phenomenon of test overfitting, where models exploit weaknesses in test coverage to produce code that passes validation without addressing the underlying logical error. Plausible patches often exhibit higher lexical similarity to the original buggy code than the actual fixed versions, suggesting that models may prioritize minimal edits that satisfy test oracles rather than engaging in deep contextual reasoning to identify specific variable misassignments or operator faults. Consequently, relying solely on test-based metrics like pass@k can inflate performance estimates, as these metrics fail to detect semantically incorrect solutions that inadvertently satisfy incomplete test specifications.

Furthermore, the reliance on exact match metrics for correctness assessment introduces construct validity threats, as multiple non-equivalent

but acceptable fixes may exist for a single bug. While manual inspection and formal verification offer more rigorous validation, they are often infeasible at scale, forcing researchers to depend on imperfect proxies. The gap between plausibility and correctness underscores the necessity for advanced evaluation frameworks that go beyond binary test outcomes, incorporating semantic analysis and human expert review to distinguish between superficially successful patches and those that genuinely resolve the root cause of software defects.

4.2.2 Cost-Efficiency and Token Usage Analysis

Cost-efficiency in Automated Program Repair (APR) is intrinsically linked to token consumption, as large language models operate under strict context window constraints and pricing structures based on input-output volume [1]. The sheer scale of debugging artifacts, including error stacks and method bodies, often exceeds standard limits, necessitating aggressive truncation or abstraction strategies. For instance, replacing specific identifiers and literals with reusable categorical IDs can reduce token counts by approximately 12%, while summarizing loop iterations with ellipses further mitigates overhead without sacrificing semantic integrity.

The economic implications of token usage scale directly with patch complexity, where intricate repairs demand extensive reasoning chains and larger context windows. Empirical analyses indicate that as repair types evolve from simple syntax fixes to complex logical restructuring, both input and output token counts increase proportionally, leading to higher monetary costs and latency. Consequently, selecting model architectures involves a critical trade-off; while massive models offer superior reasoning capabilities for difficult bugs, smaller 6-7B parameter models often provide an optimal balance between inference cost and effectiveness for routine repair tasks.

Furthermore, evaluation methodologies must account for the inefficiency of autoregressive generation, which remains a bottleneck for long-code synthesis compared to deterministic syntax checking. Strategies such as capability-based filtering are employed to exclude problems solvable by cost-effective frontier models, ensuring that computational resources are reserved for instances requiring substantive reasoning. Ultimately, opti-

mizing token usage through enhanced tokenization schemes and prompt engineering is imperative for deploying scalable APR systems that maintain high success rates without incurring prohibitive operational expenses.

4.2.3 Robustness Against Semantic Perturbations

Current neural program repair models frequently exhibit a conservative bias when facing semantic uncertainty, prioritizing syntactic fidelity over functional correction [10]. When the probability distribution over potential repairs is diffuse, autoregressive mechanisms tend to reproduce high-confidence input tokens rather than risk introducing low-probability corrective sequences. This behavior results in outputs that are grammatically valid but semantically inert, effectively preserving the original bug rather than resolving it. Consequently, while syntax error rates in generated patches have decreased, the capacity to execute meaningful semantic transformations remains critically limited across various evaluation checkpoints.

The fragility of these models becomes particularly evident under semantic perturbations, where minor, meaning-preserving code variations cause disproportionate performance degradation. Empirical benchmarks utilizing semantics-preserving transformations reveal that even state-of-the-art large language models suffer significant drops in repair accuracy when confronted with altered identifier names, reordered statements, or modified control structures that do not affect program logic [19]. This sensitivity indicates that current architectures often rely on superficial syntactic patterns rather than developing a deep, abstract understanding of program semantics, rendering them unstable in diverse coding contexts.

Furthermore, the inability to generalize across semantic variations underscores a fundamental gap between syntactic validity and logical correctness. Although models can consistently generate compilable code, they frequently fail to address the underlying logical defects, achieving near-zero exact match accuracy on complex semantic bugs. This disconnect suggests that optimizing for token-level prediction loss is insufficient for robust program repair. Future advancements must therefore shift focus from mere grammatical correctness to enhancing semantic reasoning capabilities, ensuring that repair strategies remain effective regard-

less of non-essential syntactic fluctuations in the source code.

4.3 Comparative Studies and Baseline Analysis

4.3.1 Large Language Models versus Traditional APR Tools

Traditional Automated Program Repair (APR) tools predominantly rely on heuristic search strategies, constraint solving, or probabilistic models trained on limited syntactic patterns [1]. These methodologies often exhibit inherent rigidity, struggling to generalize across diverse defect types or effectively leverage semantic context beyond immediate code surroundings. While approaches like Phog utilize probabilistic modeling for code prediction, they frequently lack the deep contextual understanding required to interpret complex natural language specifications or review comments, limiting their adaptability to broader spectra of software faults.

In contrast, Large Language Models (LLMs) represent a paradigm shift by treating repair as a generative task grounded in massive pre-training on both natural and programming languages [23]. Models such as Codex and CodeT5 demonstrate an exceptional ability to comprehend buggy code as a high-quality draft, generating syntactically valid patches with high probability even without task-specific fine-tuning. This zero-shot or few-shot capability allows LLMs to internalize intricate grammatical structures and semantic relationships, enabling them to propose fixes that traditional template-based or search-based systems often miss, particularly for bugs requiring multi-statement modifications.

$$\mathcal{L}_{\text{Gen}} = - \sum_{t=1}^T \log P_{\pi}(\hat{y}_t | \mathbf{x}, \hat{\mathbf{y}}_{<t})$$

Figure 22: Chart from 'VeriDebug: A Unified LLM for Verilog Debugging via Contrastive Embedding and Guided Correction' (arXiv:2504.19099)

However, the superiority of LLMs comes with significant computational trade-offs regarding model size, training costs, and inference latency. While compact models like CodeT5-small offer viability for consumer hardware with impressive syntax validity, larger models generally correlate with superior out-of-distribution performance

and reduced optimization steps. Consequently, the landscape is evolving towards hybrid strategies that combine the generative power of LLMs with traditional validation engines or plastic surgery hypotheses, aiming to balance the high success rates of massive models against the efficiency and precision constraints of practical software engineering environments.

4.3.2 Impact of Prompt Engineering Variations

Prompt engineering variations exert a decisive influence on the bug-fixing efficacy of Large Language Models, often determining the boundary between successful repair and complete failure [24]. Empirical evidence indicates that models exhibit extreme sensitivity to prompt formatting, where minor structural adjustments can drastically alter output distributions. In suboptimal configurations, models frequently default to reproducing buggy inputs verbatim, with studies reporting up to eighty percent of predictions remaining unchanged from the source code [10]. Conversely, strategically designed prompts that incorporate explicit instructions or few-shot examples can elevate repair success rates by over two hundred percent, particularly for models with weaker baseline capabilities.

The specific composition of the prompt, including the inclusion of reviewer comments, bug type specifications, and repair examples, serves as a critical mechanism for guiding the model's internal reasoning. Prompts that merely request code correction without contextual hints often force the model to rely exclusively on pretraining knowledge, leading to superficial changes or hallucinated fixes. In contrast, instruction-style prompts that clearly define the task hierarchy and provide concrete examples enable the model to leverage historical heuristics more effectively. This approach not only improves the precision of generated patches but also mitigates the risk of introducing security vulnerabilities or logical errors that stem from a lack of domain-specific context.

Furthermore, the interaction between prompt design and model architecture reveals that no single prompting strategy universally optimizes performance across all datasets and programming languages. While certain prompt templates demonstrate robustness across diverse bug types, others show significant variance depending on the underlying backbone LLM and the complexity of the target method [25]. Consequently, achiev-

ing optimal automated program repair requires a systematic exploration of prompt variants rather than reliance on a static template [26]. Future research must prioritize the development of adaptive prompting frameworks that dynamically select or construct prompts based on code characteristics, thereby maximizing repair accuracy while minimizing the computational costs associated with extensive trial-and-error iterations.

$$\begin{cases} llp^*(V_b) + (1 - p^*)(-C_b) - H_c & \text{if } D_v(P_v(b, t)) \\ -C_b & \text{otherwise} \end{cases}$$

Figure 23: Chart from 'Abstain and Validate: A Dual-LLM Policy for Reducing Noise in Agentic Program Repair' (arXiv:2510.03217)

```
// cpp_peglib-1 in BugsCpp
↑ 54 lines
if (log) {
  3 lines
+   ret = false;
}
  if (!ret)
↓ 105 lines
```

Figure 24: Chart from 'Empirical Evaluation of Large Language Models in Automated Program Repair' (arXiv:2506.13186)

4.3.3 Human Developer Performance Comparisons

Comparing automated repair tools against human developers reveals a fundamental divergence in evaluation metrics and repair strategies. While automated systems often prioritize exact match scores or syntax validity, human efficacy is better measured by perceived usefulness and the ability to leverage contextual debugging aids. Studies indicate that string similarity metrics like edit distance or BLEU frequently misalign with developer needs, as humans often introduce additional information to facilitate understanding rather than minimizing character-level changes. Consequently, a patch with higher edit distance may be rated more useful by practitioners if it provides clearer logical corrections or debugging insights compared to a minimal, syntactically correct alteration.

The operational dynamics of bug fixing further distinguish human performance from current large language model capabilities [10]. Human developers typically employ an iterative approach, building upon knowledge from previous failed attempts to refine subsequent patches, whereas many automated tools generate candidates independently without such temporal context. This iterative reasoning allows humans to navigate complex, multi-method bugs more effectively, particularly when test suites are incomplete or absent. In scenarios lacking robust oracle mechanisms, human intuition compensates for missing execution feedback, a capability that purely test-suite-driven automated approaches often lack, leading to higher rates of test-suite adequate but semantically incorrect patches in machine-generated solutions.

Empirical evaluations suggest that while advanced models can match or exceed human fix rates on constrained benchmarks like QuixBugs, they struggle to generalize this performance to real-world, complex codebases without extensive sampling [27]. Human experts demonstrate superior efficiency in locating faults and synthesizing fixes within limited attempts, whereas models often require generating thousands of candidates to achieve comparable success rates. Furthermore, the integration of natural language instructions and error traces significantly narrows this gap, yet the human ability to comprehend high-level intent and apply abstract reasoning remains a critical advantage. Future research must therefore focus on bridging this cognitive gap by incorporating iterative feedback loops and improving the semantic alignment of generated patches with developer expectations.

Model	float32	float16	int8	int4
Phi-3 3.8B	35	38(+3)	37(+2)	36(+1)
Phi-3 7B		39	39	39
Phi-3 14B			39	37
Qwen2.5-Coder-0.5B-Instruct	27	30(+3)	28(+1)	25(-2)
Qwen2.5-Coder-1.5B-Instruct	38	34(-4)	34(-4)	32(-6)
Qwen2.5-Coder-3B-Instruct	38	38(±0)	38(±0)	0(-38)
Qwen2.5-Coder-7B-Instruct		38	39	38
Qwen2.5-Coder-14B-Instruct			38	38
Qwen2.5-Coder-32B-Instruct				35
Ave.diff	-	+0.5	-0.25	-11.25

Figure 25: Chart from 'How Small is Enough? Empirical Evidence of Quantized Small Language Models for Automated Program Repair' (arXiv:2508.16499)

5 Future Directions

Despite significant advancements in agentic and iterative debugging frameworks, several critical limitations persist that hinder the widespread adoption of Large Language Model-based automated program repair. Current multi-agent architectures often suffer from prohibitive computational costs and latency due to extensive interaction rounds required for consensus, making them impractical for real-time development environments. Furthermore, existing feedback loops frequently struggle with error propagation, where flawed initial hypotheses or misleading execution traces reinforce incorrect reasoning paths rather than correcting them. There is also a notable deficiency in handling large-scale, multi-file projects where context retrieval remains imperfect, leading to patches that are locally correct but globally inconsistent with broader project semantics. Finally, the lack of standardized evaluation metrics specifically designed for agentic workflows complicates the objective comparison of different architectural strategies and obscures the true trade-offs between autonomy, cost, and repair accuracy.

To address these challenges, future research must prioritize the development of efficient, adaptive arbitration protocols that dynamically balance verification rigor with resource utilization. Instead of relying on fixed interaction depths, next-generation systems should employ confidence-aware termination criteria that halt agent debates once a stability threshold is reached, significantly reducing token consumption without compromising solution quality. Concurrently, there is a pressing need to enhance the robustness of hypothesis generation mechanisms against non-deterministic model outputs. This could be achieved by integrating formal verification tools directly into the iterative loop, providing mathematical guarantees for critical code sections alongside probabilistic LLM reasoning. Additionally, future work should focus on creating holistic context management strategies that leverage knowledge graphs to maintain a coherent understanding of repository-wide dependencies, ensuring that generated patches align with long-term project evolution and architectural constraints.

The successful realization of these research directions promises to fundamentally transform software maintenance by establishing reliable, scalable, and truly autonomous debugging agents. By

mitigating computational overhead and enhancing semantic consistency, advanced agentic frameworks will enable the deployment of continuous repair systems capable of handling complex, real-world defects with minimal human intervention. This shift will not only accelerate software release cycles and reduce technical debt but also elevate the role of human developers from routine bug fixers to high-level system architects. Ultimately, bridging the gap between current experimental prototypes and production-grade tools will foster a new era of self-healing software ecosystems, significantly improving system reliability and security across diverse application domains.

6 Conclusion

This survey has systematically examined the transformative impact of Large Language Models on Automated Program Repair, highlighting a decisive shift from monolithic, single-shot generation toward sophisticated agentic and iterative frameworks. Our analysis reveals that decomposing repair tasks through role-specialized multi-agent architectures significantly mitigates cognitive overload, allowing distinct agents to manage fault localization, root cause analysis, and patch synthesis with greater precision. Furthermore, the integration of consensus mechanisms and hierarchical arbitration strategies has proven essential in reducing hallucinations, although these approaches introduce notable trade-offs between verification rigor and computational cost. Crucially, the adoption of execution-guided self-correction cycles and dynamic hypothesis generation represents a paradigm change from blind search to evidence-based exploration. By leveraging runtime feedback and test-driven constraints, modern systems can iteratively refine patches to address complex logical errors that traditionally stymied static analysis tools. Finally, the synthesis of repository-aware context management and retrieval-augmented generation ensures that generated fixes maintain semantic consistency across large-scale, multi-file projects, addressing a longstanding limitation in prior repair methodologies.

The significance of this work lies in its provision of a unified taxonomy that categorizes the rapidly expanding landscape of LLM-based debugging approaches, distinguishing clearly between architectural patterns, feedback mechanisms, and context strategies. Unlike previous re-

views that focused primarily on benchmark performance or model architectures, this survey critically analyzes the systemic workflows that enable autonomous debugging agents to function reliably in real-world scenarios. By elucidating the interplay between agent autonomy, human oversight, and iterative refinement, we offer a foundational reference for understanding how modular designs enhance transparency and controllability. This structured perspective not only clarifies the current state-of-the-art but also exposes the inherent tensions between scalability and accuracy, providing researchers and practitioners with the analytical tools necessary to navigate the design space of next-generation repair systems.

Looking forward, the field must address several persistent challenges to realize the full potential of intelligent debugging systems. Future research should prioritize the development of efficient arbitration protocols that reduce the latency and financial overhead associated with multi-agent consensus without compromising fix reliability. There is an urgent need for robust methods to handle the non-deterministic nature of model outputs, ensuring that debugging workflows remain stable across diverse codebases and evolving dependencies. Moreover, the community must establish standardized evaluation metrics specifically tailored for agentic workflows, moving beyond simple pass rates to assess factors such as reasoning quality, resource efficiency, and human-AI collaboration effectiveness. We call upon researchers to focus on building scalable, verifiable, and transparent frameworks that seamlessly integrate human expertise with autonomous agents, ultimately fostering a new era of software engineering where AI serves as a trustworthy partner in maintaining software quality and reliability.

References

- [1] Studying and Understanding the Effectiveness and Failures of Conversational LLM-Based Repair
- [2] Extending the Frontier of ChatGPT Code Generation and Debugging
- [3] Adversarial Reasoning for Repair Based on Inferred Program Intent
- [4] Debugging with Open-Source Large Language Models An Evaluation
- [5] Explainable Automated Debugging via Large Language Model-driven Scientific Debugging
- [6] Improving LLM-Assisted Secure Code Generation through Retrieval-Augmented-Generation and Multi-Tool Feedback
- [7] SWE -Synth Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs
- [8] VIBEPASS Can Vibe Coders Really Pass the Vibe Check
- [9] Mutation Testing via Iterative Large Language Model-Driven Scientific Debugging
- [10] Syntax Is Not Enough An Empirical Study of Small Transformer Models for Neural Code Repair
- [11] Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs
- [12] A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation
- [13] Revisiting Unnaturalness for Automated Program Repair in the Era of Large Language Models
- [14] MUFIN Improving Neural Repair Models with Back-Translation
- [15] A Survey on Automated Program Repair Techniques
- [16] RePair Automated Program Repair with Process-based Feedback
- [17] Enhancing Automated Program Repair via Faulty Token Localization and Quality-Aware Patch Refinement
- [18] TraceFixer Execution Trace-Guided Program Repair
- [19] Identifying and Mitigating API Misuse in Large Language Models
- [20] Automated Code Editing with Search-Generate-Modify
- [21] Hotfixing Large Language Models for Code
- [22] On the Efficiency of Test Suite based Program Repair
- [23] VeriDebug A Unified LLM for Verilog Debugging via Contrastive Embedding and Guided Correction
- [24] The Right Prompts for the Job Repair Code-Review Defects with Large Language Model
- [25] Abstain and Validate A Dual-LLM Policy for Reducing Noise in Agentic Program Repair
- [26] Empirical Evaluation of Large Language Models in Automated Program Repair
- [27] How Small is Enough Empirical Evidence of Quantized Small Language Models for Automated Program Repair